

Comprehensive Exam-Oriented Study Guide

DATA AND VISUAL ANALYTICS (AL-603-B)

UNIT IV

Data Ecosystem

B.Tech, VI Semester | RGPV Examination Scheme (GS)

DATA AND VISUAL ANALYTICS (AL-603-B)

Syllabus Topics Covered in This Unit
Overview of the Data Analyst Ecosystem
Types of Data
Understanding Different Types of File Formats
Sources of Data
Overview of Data Repositories
NoSQL
Data Marts
Data Lakes
ETL and Data Pipelines
Foundations of Big Data
Hadoop, HDFS, Hive, and Spark

Table of Contents

Unit Overview & Syllabus Map

1. Overview of the Data Analyst Ecosystem
2. Types of Data
3. Understanding Different Types of File Formats
4. Sources of Data
5. Overview of Data Repositories
6. NoSQL
7. Data Marts
8. Data Lakes
9. ETL and Data Pipelines
10. Foundations of Big Data
11. Hadoop
12. Hadoop Distributed File System (HDFS)
13. Hive
14. Apache Spark
15. Exam Focus & Quick Revision

Unit Overview & Syllabus Map

Unit IV steps back from statistics and wrangling techniques to look at the bigger infrastructure picture: where does data actually live, in what shapes, at what scale, and using which tools does an analyst pull it out and process it? This unit is the broadest in the syllabus, spanning everything from basic file formats to full Big Data ecosystems like Hadoop and Spark. It is also one of the most consistently tested units, since RGPV frequently asks 'explain X in detail' style questions on nearly every named tool and concept.

Official Syllabus Text

DEFINITION

Data Ecosystem: Overview of the Data Analyst Ecosystem, Types of Data, Understanding Different Types of File Formats, Sources of Data, Overview of Data Repositories, NoSQL, Data Marts, Data Lakes, ETL, and Data Pipelines, Foundations of Big Data, Big Data processing tools such as Hadoop, Hadoop Distributed File System (HDFS), Hive, and Spark.

LEARNING OBJECTIVES — By the end of this unit, you should be able to:

- ✓ Describe the full data analyst ecosystem, from data sources through to visualization and decision-making.
- ✓ Classify data as structured, semi-structured, or unstructured, and choose an appropriate file format for a given use case.
- ✓ Differentiate a Data Warehouse, Data Mart, and Data Lake, and explain when to use NoSQL over a relational database.
- ✓ Explain the ETL process and distinguish it from ELT and from a general data pipeline.
- ✓ Describe the foundations of Big Data (the 5 V's) and explain why distributed computing is required at that scale.
- ✓ Explain the architecture and working of Hadoop, HDFS, Hive, and Spark, and compare them meaningfully against each other.

1. Overview of the Data Analyst Ecosystem

1.1 Introduction

Before diving into individual tools and formats, it helps to see the whole landscape a data analyst operates within — the full 'ecosystem' of roles, tools, and processes that take raw data all the way to business decisions. This topic sets the mental map for the rest of Unit IV.

1.2 Definition

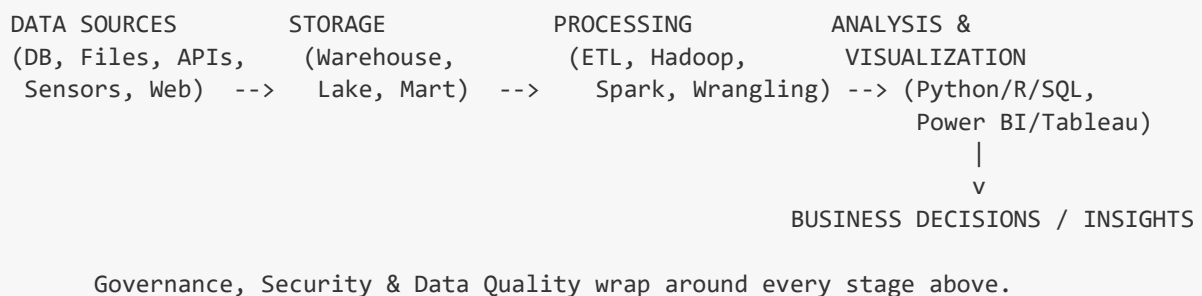
DEFINITION

Data Analyst Ecosystem: The complete set of interconnected roles, tools, technologies, processes, and data sources that together support the collection, storage, processing, analysis, and communication of data-driven insights within an organization.

1.3 Key Components of the Ecosystem

- **Data Sources:**
- Databases, files, APIs, sensors, and external/public data feeding into the system.
- **Data Storage:**
- Databases, data warehouses, data lakes, and data marts that hold the collected data.
- **Data Processing/Integration:**
- ETL pipelines and Big Data tools (Hadoop, Spark) that clean, transform, and combine data.
- **Analysis Tools:**
- Statistical/programming tools like Python (pandas, NumPy), R, SQL, and Excel.
- **Visualization & BI Tools:**
- Power BI, Tableau, Python visualization libraries (matplotlib, seaborn, plotly) that communicate insights.
- **People/Roles:**
- Data Analysts, Data Engineers, Data Scientists, Business Intelligence Developers, and Business Stakeholders who consume insights.
- **Governance & Security:**
- Policies and tools ensuring data quality, privacy, and compliance across the ecosystem.

1.4 Data Analyst Ecosystem Diagram



1.5 Key Roles in the Ecosystem (Frequently Asked as a Sub-Point)

Role	Primary Responsibility
Data Analyst	Explores, analyzes, and visualizes data to answer specific business questions
Data Engineer	Builds and maintains the pipelines and infrastructure that move and store data
Data Scientist	Builds predictive models and applies advanced statistics/machine learning
BI Developer	Builds dashboards and reports for ongoing business monitoring
Database Administrator (DBA)	Manages database performance, security, and availability

1.6 Working Process — How Data Flows Through the Ecosystem

1. Data is generated/captured at various sources (transactions, sensors, forms, third-party feeds).
2. Data is ingested and stored in an appropriate repository (database, data lake, warehouse) based on its structure and volume.
3. ETL/data pipeline processes clean, transform, and integrate the data (Unit III wrangling techniques apply here).
4. For very large volumes, Big Data tools (Hadoop, Spark) process the data in a distributed manner.
5. Analysts query and explore the processed data using SQL, Python, or R.
6. Insights are communicated to stakeholders via visualizations, dashboards, and reports.
7. Business decisions are made, which in turn often generate new data — completing the cycle.

1.7 Advantages of Understanding the Full Ecosystem

- Helps an analyst choose the right tool for each stage of a project rather than a one-size-fits-all approach.
- Improves collaboration with data engineers and scientists by understanding their role and handoffs.
- Provides context for why certain data quality or governance rules exist.

1.8 Limitations / Challenges

- The ecosystem is constantly evolving with new tools, requiring continuous learning.
- Different organizations may implement very different combinations of these components, making generic knowledge insufficient without contextual adaptation.
- Tool sprawl (too many overlapping tools) can create integration and governance challenges.

1.9 Applications

- Onboarding new analysts to understand where their work fits into a larger data strategy.
- Designing end-to-end analytics architecture for a new organization or project.
- Career planning — understanding adjacent roles (data engineer, data scientist) an analyst may grow into.

MEMORY TIP

Ecosystem memory tip: 'SSPAV' — Sources, Storage, Processing, Analysis, Visualization — the five pillars of the data analyst ecosystem, wrapped in Governance.

EXAM TIP

For 'Explain about Data Analyst Ecosystem' (asked May 2023), always draw the source→storage→processing→analysis→visualization flow diagram and name at least 3 tools per stage — this single diagram format is exactly what RGPV examiners expect.

AL-603-B

2. Types of Data

2.1 Introduction

Beyond the statistical variable types covered in Unit I (nominal, ordinal, etc.), Unit IV looks at data types from a systems/storage perspective — how data is organized at a structural level, which directly determines what storage technology and processing tool is appropriate.

2.2 The Three Structural Types of Data

Type	Definition	Storage Fit	Examples
Structured Data	Data organized in a fixed, predefined schema — rows and columns with defined data types	Relational databases (SQL), spreadsheets	Sales transaction tables, employee records
Semi-Structured Data	Data that does not fit a rigid table structure but still has organizational tags/markers giving it some structure	NoSQL databases, document stores	JSON, XML, YAML, email headers, log files
Unstructured Data	Data with no predefined structure or organization	Data lakes, object/file storage	Images, videos, audio, free-text documents, social media posts

2.3 Diagram — Structural Data Spectrum

STRUCTURED <-----> SEMI-STRUCTURED <-----> UNSTRUCTURED
 (Rigid schema, SQL tables) (Tags/keys but no fixed schema, JSON/XML) (No fixed schema, free text/media)

Increasing storage/processing flexibility, decreasing ease of direct querying -->

2.4 Other Classifications of Data (Additional Depth)

- By volume/velocity/variety:
- Traditional data vs Big Data (covered in Section 10).
- By origin:
- Internal/organizational data vs external/third-party data.
- By update frequency:
- Static (historical) data vs streaming (real-time) data.

2.5 Working Process — Deciding How to Store a Given Dataset

1. Check whether the data naturally fits fixed rows and columns with a consistent schema → if yes, structured, use a relational database.
2. If the data has some organization (keys, tags, nesting) but the schema can vary between records → semi-structured, use a NoSQL/document store.

3. If the data has no inherent tabular or tagged organization at all → unstructured, use a data lake or object storage.

2.6 Advantages / Disadvantages by Type

Type	Advantages	Disadvantages
Structured	Easy to query (SQL), strong consistency, mature tooling	Rigid schema is hard to change; poor fit for varied/complex data
Semi-Structured	Flexible schema, good for evolving data models	More complex to query than pure tables; some redundancy
Unstructured	Extremely flexible, captures rich real-world content (text/media)	Hard to search/query directly; requires specialized processing (NLP, computer vision) to extract value

2.7 Applications

- Structured: financial transaction systems, inventory management.
- Semi-structured: web APIs, configuration files, IoT sensor payloads.
- Unstructured: social media analytics, medical imaging, customer support call recordings.

MEMORY TIP

Types of data memory tip: 'SSU' ladder — Structured (tables), Semi-structured (tags), Unstructured (no structure) — increasing flexibility, decreasing queryability as you go down the ladder.

3. Understanding Different Types of File Formats

3.1 Introduction

File format knowledge is deceptively important — choosing the wrong format can make a Big Data pipeline dramatically slower or a dataset impossible to share cleanly. This topic has appeared as a direct 7-mark question in BOTH available papers, making it one of the highest-priority topics in the entire unit.

3.2 Definition

DEFINITION

File Format: A standardized way of encoding and structuring data within a file so that it can be reliably created, read, and interpreted by different software systems.

3.3 Text-Based / Common Data File Formats

Format	Description	Best For
CSV (Comma-Separated Values)	Plain text, one row per line, values separated by commas	Simple tabular data exchange; universally supported
TSV (Tab-Separated Values)	Like CSV but uses tabs as delimiters	Data where values may contain commas
JSON (JavaScript Object Notation)	Lightweight, human-readable, key-value/nested structure	Web APIs, semi-structured/nested data
XML (eXtensible Markup Language)	Tag-based markup format with strict hierarchical structure	Configuration files, legacy enterprise data exchange, SOAP APIs
YAML	Human-friendly data serialization format, often used for configuration	Configuration files, DevOps/infrastructure-as-code
TXT	Plain unstructured text	Free-form notes, logs, unstructured documents

3.4 Spreadsheet / Office Formats

- XLSX/XLS:
- Microsoft Excel workbook formats, support formulas, multiple sheets, formatting.
- ODS:
- OpenDocument Spreadsheet, open-standard alternative to XLSX.

3.5 Big Data / Columnar File Formats (High-Value for Exams)

Format	Storage Model	Key Advantage	Typical Use
Parquet	Columnar	Highly compressed, very fast for analytical (column-based) queries	Big Data analytics on Hadoop/Spark
ORC (Optimized Row Columnar)	Columnar	Optimized specifically for Hive; strong compression and indexing	Hive-based data warehousing

Format	Storage Model	Key Advantage	Typical Use
Avro	Row-based	Compact binary format with embedded schema; great for streaming/serialization	Data serialization in Kafka pipelines, schema evolution
SequenceFile	Row-based (binary)	Native Hadoop format for key-value pairs	Intermediate data in Hadoop MapReduce jobs

3.6 Row-Based vs Columnar Storage (Important Conceptual Comparison)

ROW-BASED (e.g., CSV, Avro): Row1: [id, name, age, salary] Row2: [id, name, age, salary] Row3: [id, name, age, salary] Best for: reading/writing FULL records (transactional workloads)	COLUMNAR (e.g., Parquet, ORC): Column 'id': [1, 2, 3, ...] Column 'name': [A, B, C, ...] Column 'age': [25, 30, 41, ...] Best for: analytical queries that scan only a FEW columns (e.g., AVG(salary))
--	---

3.7 Other Common Formats Worth Naming

- HTML:
- Markup format for web pages, sometimes scraped as a data source.
- PDF:
- Portable Document Format, common for reports; not directly analysis-friendly without extraction.
- Image/Audio/Video formats (JPEG, PNG, MP3, MP4):
- Store unstructured multimedia data.
- HDF5:
- Hierarchical Data Format, used for large scientific/numerical datasets.

3.8 Working Process — Choosing the Right File Format

1. Determine the data's structure (structured/semi-structured/unstructured) from Section 2.
2. Determine the primary access pattern: reading full records (row-based) vs analytical aggregation over a few columns (columnar).
3. Consider scale: for Big Data volumes, prefer compressed, splittable formats like Parquet/ORC/Avro over plain CSV/JSON.
4. Consider interoperability: choose widely-supported formats (CSV, JSON) when sharing with external, less specialized systems.
5. Consider schema evolution needs: Avro is preferred when the schema is expected to change over time.

3.9 Advantages of Using the Right File Format

- Columnar formats (Parquet/ORC) drastically reduce I/O and speed up analytical queries on Big Data platforms.
- Formats with embedded schema (Avro, Parquet) improve data reliability and self-documentation.
- Compressed binary formats significantly reduce storage costs at scale.

3.10 Disadvantages / Limitations

- Plain text formats (CSV/JSON) are easy to use but inefficient and larger in size for Big Data volumes.
- Columnar formats are less efficient for transactional (row-by-row) read/write operations.
- Binary formats (Parquet, Avro, ORC) are not human-readable, complicating manual debugging.

3.11 Applications

- CSV/JSON: everyday data exchange, APIs, quick exports.
- Parquet/ORC: data warehousing and analytical big-data queries (Hive, Spark SQL).
- Avro: streaming data pipelines (Kafka) needing schema evolution.
- XML/YAML: configuration and legacy enterprise system integration.

MEMORY TIP

File format memory tip: 'CJXY for everyday, PAO for Big Data' — CSV/JSON/XML/YAML for common exchange; Parquet/Avro/ORC for Big Data analytics.

EXAM TIP

For 'Describe the different types of file formats in detail' (asked in both years), structure your answer as: text-based formats table → spreadsheet formats → Big Data columnar formats table → row vs columnar diagram → 1-2 lines on choosing the right format. This covers maximum syllabus breadth in one answer.

4. Sources of Data

4.1 Introduction

This topic overlaps with 'Gathering Data' from Unit III but is examined here from an ecosystem/infrastructure perspective — WHERE data physically or systemically originates within and outside an organization, rather than HOW to collect it.

4.2 Definition

DEFINITION

Data Source: Any system, device, repository, or entity that originates or provides data for collection, storage, and subsequent analysis.

4.3 Classification of Data Sources

Category	Description	Examples
Internal Sources	Data generated within the organization's own systems	Transactional databases, CRM, ERP, internal logs, employee records
External Sources	Data obtained from outside the organization	Government open data, purchased datasets, social media, partner data feeds
Machine-Generated Sources	Data automatically produced by devices/systems	IoT sensors, server logs, application logs, GPS trackers
Human-Generated Sources	Data produced through direct human action	Surveys, forms, social media posts, emails, customer reviews

4.4 Common Real-World Data Sources

- Relational and NoSQL databases (transactional systems).
- Web APIs (weather, financial, social media platforms).
- Flat files (CSV, Excel exports from legacy systems).
- Streaming sources (Kafka topics, sensor telemetry).
- Web scraping of publicly available websites.
- Third-party data vendors and open government data portals.

4.5 Working Process — Identifying and Evaluating a Data Source

1. Identify what business question needs to be answered.
2. List candidate data sources (internal and external) that could contain relevant information.
3. Evaluate each source on: relevance, data quality, accessibility, update frequency, cost, and legal/licensing constraints.
4. Select and document the chosen source(s), along with connection/access method.

4.6 Advantages of Diverse Data Sourcing

- Combining internal and external sources produces richer, more context-aware analysis.
- Machine-generated sources provide continuous, high-frequency data ideal for real-time monitoring.

4.7 Limitations

- External sources may have inconsistent quality, licensing restrictions, or unclear provenance.
- Integrating many different source types (structured + unstructured, internal + external) adds significant wrangling complexity.

4.8 Applications

- Building a 360-degree customer view by combining CRM, support, and social media data.
- Market research combining internal sales data with external market trend reports.

MEMORY TIP

Sources memory tip: 'IEMH' — Internal, External, Machine-generated, Human-generated — the four broad categories of data sources.

5. Overview of Data Repositories

5.1 Introduction

A data repository is the general umbrella term for any place data is stored for management and analysis. Understanding the landscape of repository types is essential before diving into the specific technologies (NoSQL, Data Marts, Data Lakes) covered later in this unit.

5.2 Definition

DEFINITION

Data Repository: A centralized location — physical or virtual — where data is stored, organized, and maintained for management, access, and analysis purposes.

5.3 Major Types of Data Repositories

Repository Type	Description	Best Suited For
Database (Relational/NoSQL)	Structured or semi-structured storage optimized for transactional read/write operations	Day-to-day operational applications
Data Warehouse	Centralized, structured repository optimized for query and analysis of historical, integrated data (subject-oriented, uses schema-on-write)	Business intelligence and reporting
Data Mart	A smaller, subject/department-specific subset of a data warehouse	Focused departmental reporting (Section 7)
Data Lake	Centralized repository that stores raw data in its native format at any scale (schema-on-read)	Big Data, machine learning, unstructured/semi-structured data (Section 8)
Data Lakehouse	Hybrid architecture combining data lake flexibility with data warehouse structure/governance	Modern unified analytics platforms

5.4 Data Warehouse vs Data Lake (Very Frequently Tested Comparison)

Aspect	Data Warehouse	Data Lake
Data structure	Structured, processed data only	Structured, semi-structured, and unstructured data
Schema	Schema-on-write (defined before loading)	Schema-on-read (defined when data is read/used)
Users	Business analysts (via SQL/BI tools)	Data scientists, engineers (via code/notebooks)
Cost	Higher (structured storage, indexing)	Lower (raw, often cloud object storage)
Agility	Less agile — schema changes are costly	Highly agile — any data can be dropped in as-is

5.5 Working Process — Choosing a Repository Type

1. If data is transactional and needs fast, consistent read/write for an application → Database.
2. If data is already structured, cleaned, and needed for company-wide historical reporting → Data Warehouse.
3. If a single department needs a fast, focused subset of the warehouse → Data Mart.
4. If data is raw, high-volume, and of mixed types (including unstructured), especially for ML/exploratory work → Data Lake.

5.6 Advantages / Limitations Summary

- Advantages:
 - Centralizing data in appropriate repositories improves accessibility, consistency, and enables governance/security controls.
- Limitations:
 - Maintaining multiple repository types adds architectural complexity and potential data duplication/synchronization challenges.

5.7 Applications

- Enterprise reporting infrastructure design.
- Choosing storage architecture for a new analytics or ML initiative.

MEMORY TIP

Repository memory tip: 'DB-DW-DM-DL' — Database (operational) → Data Warehouse (structured historical) → Data Mart (departmental slice) → Data Lake (raw, all types).

6. NoSQL

6.1 Introduction

As data grew in volume and variety beyond what traditional relational databases could efficiently handle — especially semi-structured and unstructured data at massive scale — NoSQL ('Not Only SQL') databases emerged as a flexible, horizontally scalable alternative.

6.2 Definition

DEFINITION

NoSQL Database: A category of database management systems that store and retrieve data using models other than the traditional relational (tabular) model, typically offering flexible schemas and horizontal scalability suited to large volumes of structured, semi-structured, or unstructured data.

6.3 Four Major Types of NoSQL Databases

Type	Data Model	Example Systems	Best For
Document Store	Stores data as flexible, self-describing documents (usually JSON/BSON)	MongoDB, CouchDB	Content management, catalogs, semi-structured records
Key-Value Store	Simple pairs of a unique key and its value	Redis, DynamoDB	Caching, session storage, very high-speed lookups
Column-Family Store	Data organized into column families rather than rows	Cassandra, HBase	Very large-scale, write-heavy workloads (e.g., time-series, logs)
Graph Database	Data stored as nodes and relationships (edges)	Neo4j, Amazon Neptune	Social networks, recommendation engines, fraud detection

6.4 SQL vs NoSQL (High-Yield Comparison Table)

Aspect	SQL (Relational)	NoSQL
Schema	Fixed, predefined schema	Flexible / dynamic schema
Scalability	Primarily vertical (bigger server)	Primarily horizontal (more servers/nodes)
Data model	Tables with rows and columns	Document, key-value, column-family, or graph
Consistency model	Strong consistency (ACID)	Often eventual consistency (BASE) for scalability
Best for	Structured data, complex transactions/joins	Large-scale, fast-changing, semi/unstructured data

ACID = Atomicity, Consistency, Isolation, Durability (strict transactional guarantees). BASE = Basically Available, Soft state, Eventual consistency (relaxed guarantees favouring scale and availability).

6.5 Working Process — When to Choose NoSQL

1. Assess data variety: if data is highly variable in structure (e.g., different documents have different fields) → NoSQL document store fits well.

2. Assess scale: if the application needs to scale horizontally across many distributed servers → NoSQL is generally easier to scale than relational.
3. Assess consistency needs: if strict transactional (ACID) guarantees are critical (e.g., banking transactions) → prefer SQL; if eventual consistency is acceptable for higher availability → NoSQL is suitable.
4. Match the specific NoSQL type to the access pattern (lookups → key-value; relationships → graph; huge write-heavy logs → column-family; flexible records → document).

6.6 Advantages of NoSQL

- Flexible schema accommodates rapidly evolving or highly varied data structures.
- Scales horizontally across many commodity servers, well-suited to Big Data volumes.
- Often provides better performance for specific access patterns (e.g., key-value lookups).

6.7 Disadvantages / Limitations

- Generally weaker consistency guarantees compared to relational ACID transactions.
- Less mature standardized query language compared to SQL (each NoSQL system has its own query API/language).
- Joins across collections/tables are typically harder or unsupported compared to relational databases.

6.8 Applications

- MongoDB: content management systems, product catalogs.
- Redis: caching layers, real-time leaderboards, session stores.
- Cassandra: large-scale write-heavy applications (e.g., IoT telemetry, messaging systems).
- Neo4j: social network analysis, fraud detection, recommendation engines.

MEMORY TIP

NoSQL memory tip: 'DKCG' — Document, Key-value, Column-family, Graph — the four NoSQL data models.

7. Data Marts

7.1 Introduction

Not every department needs access to an organization's entire data warehouse — a Data Mart provides a smaller, faster, more focused slice of data tailored to a specific business function, improving both performance and usability for that group.

7.2 Definition

DEFINITION

Data Mart: A subset of a data warehouse, focused on a specific business line, department, or subject area (such as sales, finance, or marketing), designed to give targeted user groups faster and simpler access to relevant data.

7.3 Types of Data Marts

Type	Description
Dependent Data Mart	Built by extracting a subset of data FROM an existing central data warehouse (top-down approach)
Independent Data Mart	Built directly from operational source systems WITHOUT a central warehouse (bottom-up approach)
Hybrid Data Mart	Combines data from both a central warehouse and additional independent/external sources

7.4 Working Process — Building a Data Mart

1. Identify the specific business department/function's data and reporting needs (e.g., Sales Data Mart).
2. Identify the relevant source(s) — either the central data warehouse (dependent) or operational systems directly (independent).
3. Extract, transform, and load only the relevant subset of data into the data mart's schema.
4. Design the schema (often a star schema with fact and dimension tables) optimized for that department's typical queries.
5. Provide access to the department's analysts/BI tools, refreshing the data mart on a defined schedule.

7.5 Data Warehouse vs Data Mart

Aspect	Data Warehouse	Data Mart
Scope	Enterprise-wide, multiple subject areas	Single department/subject area
Size	Large	Smaller, more focused
Build time	Longer to design and implement	Faster to build and deploy
Users	Cross-organizational analysts	Specific department's analysts

7.6 Advantages

- Faster query performance due to smaller, focused dataset.
- Easier for departmental users to understand and use (simpler schema, relevant fields only).
- Can be implemented more quickly and cheaply than a full enterprise warehouse.

7.7 Disadvantages / Limitations

- Independent data marts built without central governance can create data silos and inconsistent definitions across departments.
- Multiple data marts may duplicate storage and ETL effort compared to one centralized warehouse.

7.8 Applications

- A Sales Data Mart giving the sales team fast access to revenue, pipeline, and customer metrics.
- A Finance Data Mart focused on budgeting, expenses, and financial reporting.

MEMORY TIP

Data Mart memory tip: 'Mart = smart & small' — a data mart is the smaller, department-smart slice of the bigger data warehouse.

8. Data Lakes

8.1 Introduction

As organizations began collecting massive volumes of raw, varied data (logs, images, JSON, sensor feeds) that didn't fit neatly into a structured warehouse, the Data Lake emerged as a storage architecture that keeps data in its native, raw format until it is actually needed for analysis.

8.2 Definition

DEFINITION

Data Lake: A centralized storage repository that holds vast amounts of raw data in its native format — structured, semi-structured, and unstructured — until it is needed, using a 'schema-on-read' approach rather than requiring a predefined schema at load time.

8.3 Key Characteristics

- Stores data 'as-is' without requiring upfront transformation or a fixed schema.
- Supports all data types: structured tables, semi-structured JSON/XML, and unstructured images/video/text.
- Uses schema-on-read: the structure is applied only when the data is read/queried, not when it is stored.
- Typically built on scalable, low-cost storage (e.g., Hadoop HDFS, Amazon S3, Azure Data Lake Storage).

8.4 Data Lake Architecture (Zones)

RAW / LANDING ZONE	-->	CLEANED / STAGING ZONE	-->	CURATED / ANALYTICS ZONE
(data as ingested, completely unmodified)		(wrangled per Unit III: cleaned, structured)		(business-ready, often aggregated tables)

8.5 Working Process — Data Lake Lifecycle

1. Ingest raw data from all sources directly into the landing zone, preserving the original format.
2. Catalog the data with metadata (source, format, ingestion date) so it remains discoverable.
3. Process/wrangle the data as needed (Unit III techniques) into a cleaned staging zone.
4. Curate business-ready datasets in the analytics zone for consumption by analysts, BI tools, or ML models.
5. Apply governance: access control, data quality monitoring, and lifecycle/retention policies throughout.

8.6 Data Lake vs Data Warehouse (Reiterated for This Section's Context)

See the detailed comparison table in Section 5.4. The key exam-friendly summary: a Data Warehouse stores processed, structured data for known questions; a Data Lake stores raw, all-format data for both known and not-yet-known future questions.

8.7 Advantages

- Extremely flexible — can store any data type without upfront modeling effort.

- Cost-effective for storing massive volumes of raw data (commodity/cloud storage).
- Ideal foundation for machine learning and exploratory data science work.

8.8 Disadvantages / Limitations

- Without careful governance, a data lake can degrade into a 'data swamp' — an unmanaged, undiscoverable, low-trust mess of raw files.
- Querying raw, unstructured data directly is slower and more complex than querying a well-modeled warehouse.
- Requires more sophisticated tooling and skilled personnel (data engineers) to extract value.

8.9 Applications

- Centralized storage for machine learning training data (images, logs, text).
- IoT sensor data storage before downstream processing.
- Long-term, low-cost archival of raw operational data for future, not-yet-defined analysis needs.

MEMORY TIP

Data Lake memory tip: 'Lake = raw, Warehouse = refined' — a lake holds water (data) in its natural raw state; a warehouse holds neatly boxed, labeled (structured) goods.

EXAM TIP

A classic short-answer trap: students confuse Data Lake and Data Warehouse. Always anchor your answer on the single clearest distinguishing fact — schema-on-read (Lake) vs schema-on-write (Warehouse).

9. ETL and Data Pipelines

9.1 Introduction

ETL is the classic, formalized process for moving data from source systems into a target repository (typically a data warehouse), and Data Pipelines are the broader, more general infrastructure concept encompassing ETL and other data-movement workflows, including modern real-time streaming scenarios.

9.2 Definitions

DEFINITION

ETL (Extract, Transform, Load): A data integration process that extracts data from one or more source systems, transforms it into a clean, consistent, analysis-ready format, and loads it into a target data repository such as a data warehouse.

DEFINITION

Data Pipeline: A broader set of automated processes and tools that move data from one or more sources to one or more destinations, applying any necessary processing steps along the way — of which ETL is one common, structured example.

9.3 The Three Stages of ETL

1. Extract: Pull raw data out of source systems (databases, files, APIs) without altering it.
2. Transform: Clean, validate, restructure, aggregate, and enrich the extracted data to match the target schema and quality requirements (this stage heavily overlaps with Unit III wrangling techniques).
3. Load: Write the transformed data into the target repository (data warehouse, data mart, or lake).

```
SOURCE SYSTEMS  --Extract-->  STAGING AREA  --Transform-->  Clean Data  --Load-->  TARGET
(DB, files, APIs)                                     (Warehouse/
                                                         Mart/Lake)
```

9.4 ETL vs ELT (Modern Variant — Good for Bonus Marks)

Aspect	ETL (Extract-Transform-Load)	ELT (Extract-Load-Transform)
Order of operations	Transform happens BEFORE loading into the target	Raw data is loaded first; transformation happens INSIDE the target system
Best fit	Traditional data warehouses with limited compute	Modern cloud data warehouses/lakes with powerful in-database compute (e.g., Snowflake, BigQuery)
Flexibility	Less flexible — transformation logic must be predefined	More flexible — raw data is preserved, transformations can be redefined later

9.5 Data Pipeline Types

- Batch Pipelines:
 - Process data in scheduled chunks (e.g., every night) — traditional ETL fits here.
- Streaming (Real-Time) Pipelines:
 - Continuously process data as it arrives, with minimal latency (e.g., using Apache Kafka + Spark Streaming).
- Hybrid Pipelines:
 - Combine batch and streaming processing for different parts of the same overall workflow.

9.6 Working Process — Designing a Data Pipeline

1. Identify source systems and the target repository, and define the required data flow.
2. Decide on batch vs streaming based on how quickly the data needs to be available for use.
3. Design the extraction method for each source (database query, API call, file watcher, streaming consumer).
4. Design transformation logic (cleaning, joining, aggregating) — ideally scripted/automated and version-controlled.
5. Design the load step into the target repository, including handling of duplicates/incremental updates.
6. Add monitoring, logging, and error-handling/retry logic to ensure pipeline reliability.
7. Schedule/orchestrate the pipeline (using tools like Apache Airflow) and monitor its ongoing execution.

9.7 Common Tools

Tool	Category
Apache NiFi	Data flow / pipeline automation
Apache Airflow	Pipeline orchestration and scheduling
Talend, Informatica	Enterprise ETL platforms
Apache Kafka	Real-time data streaming
Apache Spark	Large-scale batch and streaming transformation (Section 14)

9.8 Advantages

- Automates and standardizes the otherwise error-prone, repetitive process of moving and transforming data.
- Enables reliable, scheduled or real-time refreshing of analytics-ready data.
- Improves data quality and consistency by centralizing transformation logic.

9.9 Disadvantages / Limitations

- Complex pipelines can be fragile — a failure at one stage can break the entire downstream flow.
- Requires ongoing maintenance as source systems and business requirements change.
- Streaming pipelines are significantly more complex to design, test, and monitor than batch pipelines.

9.10 Applications

- Nightly batch ETL loading yesterday's sales data into a warehouse for morning reports.
- Real-time streaming pipeline feeding a live fraud-detection system.
- Automated pipeline feeding a machine learning model with fresh feature data.

MEMORY TIP

ETL memory tip: simply 'Extract - Transform - Load', in that exact order — but remember ELT (modern cloud approach) flips the last two steps.

10. Foundations of Big Data

10.1 Introduction

Big Data refers to datasets so large, fast-moving, or varied that traditional data processing tools (a single relational database on a single server) can no longer handle them efficiently. Understanding the foundational characteristics of Big Data explains WHY tools like Hadoop and Spark (Sections 11-14) were created in the first place.

10.2 Definition

DEFINITION

Big Data: Extremely large, complex, and/or fast-growing datasets that traditional data processing software and hardware cannot efficiently capture, store, manage, or analyze, typically requiring distributed computing techniques.

10.3 The V's of Big Data (Core Framework — Always Asked)

V	Meaning	Example
Volume	The sheer amount of data generated	Petabytes of social media data generated daily
Velocity	The speed at which data is generated and must be processed	Real-time stock market ticks, sensor streams
Variety	The many different types/formats of data	Text, images, video, structured logs, all combined
Veracity	The trustworthiness/quality/uncertainty of the data	Noisy social media data with typos, spam, bots
Value	The actual usefulness/business insight that can be extracted	Turning raw clickstream data into a personalization engine

Note: The original model proposed 3 V's (Volume, Velocity, Variety); modern treatments commonly extend this to 5 V's by adding Veracity and Value — mentioning this evolution shows extra depth in an exam answer.

10.4 Why Traditional Systems Fail at Big Data

- A single server has finite CPU, memory, and disk I/O — it cannot scale indefinitely (vertical scaling has hard physical limits).
- Relational databases assume structured data with a fixed schema, which does not fit the Variety of Big Data.
- Traditional batch processing is too slow for the real-time Velocity many modern applications require.

10.5 The Big Data Solution — Distributed Computing

DEFINITION

Distributed Computing (for Big Data): An approach where large datasets and computational workloads are split across many networked computers (a cluster), which work in parallel to store and process the data far faster and at far greater scale than any single machine could achieve.

Traditional (single server):

[One big, expensive server]

Stores + processes ALL data

-> hits a scaling ceiling

Big Data (distributed cluster):

[Node1] [Node2] [Node3] [Node4] ...

Data is SPLIT across nodes;

each node processes ITS piece in parallel

10.6 Working Process — General Big Data Processing Approach

1. Ingest data (often from a data lake) into a distributed storage system (e.g., HDFS).
2. Split ('partition') the data across many nodes in a cluster.
3. Distribute the processing task so that each node processes its own local partition of the data in parallel.
4. Combine/aggregate the partial results from all nodes into a final result.
5. This 'divide and conquer' principle underlies frameworks like MapReduce and Spark, covered in the following sections.

10.7 Advantages of Big Data & Distributed Processing

- Enables analysis at scales completely impossible on a single machine.
- Horizontal scaling (adding more commodity nodes) is often cheaper than vertical scaling (buying ever-bigger servers).
- Enables real-time or near-real-time insights from high-velocity data streams.

10.8 Disadvantages / Challenges

- Significantly more complex infrastructure to set up, secure, and maintain than a single-server system.
- Distributed systems introduce new failure modes (network partitions, node failures) that must be explicitly handled.
- Requires specialized skills (distributed systems, cluster tools) that are less common than traditional database skills.

10.9 Applications

- Social media platforms analyzing billions of daily interactions.
- E-commerce recommendation engines processing massive clickstream data.
- Genomics research processing terabytes of sequencing data.
- Financial institutions detecting fraud in real-time across millions of transactions.

MEMORY TIP

Big Data memory tip: '5 V's — Volume, Velocity, Variety, Veracity, Value' — recite them in this order every time; examiners often just want this list explained.

11. Hadoop

11.1 Introduction

Apache Hadoop is the foundational open-source framework that made practical, affordable Big Data processing possible, by combining distributed storage (HDFS) with a distributed processing model (MapReduce) running on clusters of ordinary, low-cost commodity hardware.

11.2 Definition

DEFINITION

Apache Hadoop: An open-source framework that enables the distributed storage and processing of very large datasets across clusters of commodity computers, using a simple programming model, and designed to scale from a single server to thousands of machines while automatically handling hardware failures.

11.3 Core Components of the Hadoop Ecosystem

Component	Role
HDFS (Hadoop Distributed File System)	Distributed storage layer — stores massive files split across the cluster (Section 12)
MapReduce	The original distributed processing/programming model for Hadoop (Map phase + Reduce phase)
YARN (Yet Another Resource Negotiator)	Cluster resource management layer — allocates CPU/memory to jobs across the cluster
Hadoop Common	Shared libraries and utilities used by all other Hadoop modules

11.4 The MapReduce Programming Model

MapReduce processes large datasets by breaking a job into two phases:

- Map phase:
- Each node processes its local chunk of data independently, producing intermediate key-value pairs.
- Shuffle & Sort:
- Intermediate results with the same key are grouped together across the cluster.
- Reduce phase:
- Each group of values sharing a key is aggregated/combined into the final output.

Example: Word Count across a huge text corpus

Input split across nodes -> MAP: each node emits (word, 1) for every word it sees
 -> SHUFFLE: all (word, 1) pairs for the same word are grouped
 -> REDUCE: sum the 1's for each word -> final (word, total_count)

11.5 Working Process — How a Hadoop Job Runs

1. Large input data is stored, already split into blocks, across HDFS on many DataNodes.
2. YARN allocates cluster resources (CPU, memory) to the submitted job.

3. The Map phase runs in parallel on each node, operating on the locally stored data block ('data locality' — moving computation to the data, not the other way around).
4. Intermediate key-value results are shuffled and sorted across the cluster.
5. The Reduce phase aggregates the sorted intermediate results into the final output.
6. The final output is written back to HDFS.

11.6 Key Design Principles

- Data Locality:
- Move computation TO the data (rather than moving huge data across the network to a central processor) — a core efficiency principle.
- Fault Tolerance:
- Data is automatically replicated across multiple nodes (default 3x in HDFS), so node failures do not cause data loss.
- Horizontal Scalability:
- New commodity nodes can be added to the cluster to increase capacity linearly.
- Schema-on-read:
- Hadoop does not require data to be structured/modeled before storage.

11.7 Advantages

- Cost-effective — runs on inexpensive commodity hardware rather than specialized expensive servers.
- Highly scalable and fault-tolerant, suited to petabyte-scale data.
- Open-source with a large ecosystem of complementary tools (Hive, Pig, HBase, Spark).

11.8 Disadvantages / Limitations

- MapReduce is disk-based and relatively slow for iterative or real-time workloads (this is a key reason Spark, Section 14, was developed).
- Complex to set up, tune, and administer compared to a single traditional database.
- Not efficient for small datasets — the overhead of distributed processing outweighs the benefit at small scale.

11.9 Applications

- Large-scale log processing and analysis.
- Batch processing of massive historical datasets for reporting.
- As the storage/foundation layer underpinning higher-level tools like Hive and Spark.

MEMORY TIP

Hadoop memory tip: 'HMY-C' — HDFS (storage), MapReduce (processing), YARN (resource management), Common (shared utilities) — the four core Hadoop modules.

12. Hadoop Distributed File System (HDFS)

12.1 Introduction

HDFS is Hadoop's storage layer and one of the most frequently examined topics in this unit (asked directly for 7 marks in May 2023). It is purpose-built to reliably store extremely large files across a cluster of commodity machines.

12.2 Definition

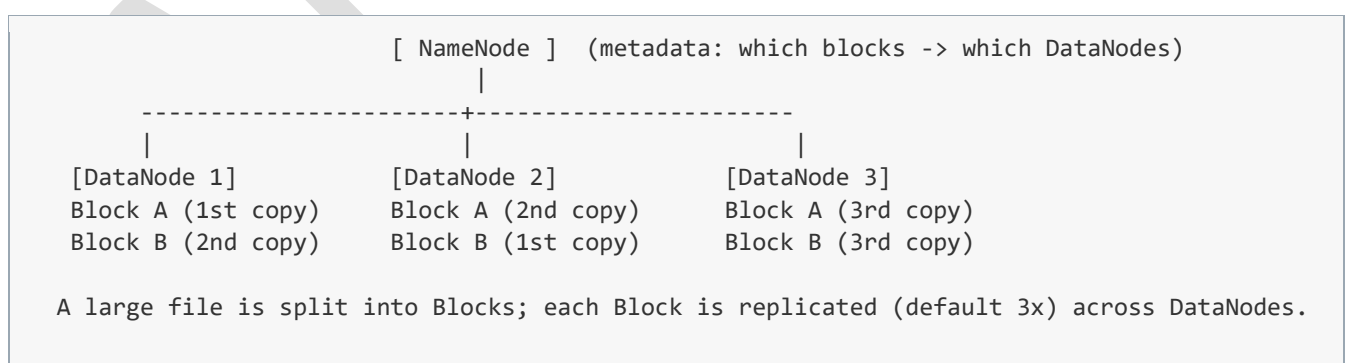
DEFINITION

HDFS (Hadoop Distributed File System): A distributed, fault-tolerant file system designed to store very large files reliably across clusters of commodity hardware, by splitting files into large blocks and replicating each block across multiple machines.

12.3 HDFS Architecture — Master/Slave Model

Component	Role
NameNode (Master)	Stores metadata only — the file system namespace, directory structure, and which DataNodes hold which blocks. Does NOT store actual data.
DataNode (Slave/Worker)	Stores the actual data blocks on local disk, and serves read/write requests directly to clients.
Secondary NameNode	Periodically merges/checkpoints the NameNode's metadata logs — NOT a hot backup/failover NameNode (a common misconception).
Block	The basic unit of storage in HDFS — files are split into large blocks (traditionally 128 MB, configurable), each replicated across multiple DataNodes.

12.4 HDFS Architecture Diagram



12.5 Key Features of HDFS

- Block replication:
- Each block is stored on multiple (default 3) DataNodes, ensuring no data is lost even if a node fails.
- Write-once-read-many:
- HDFS is optimized for large files written once and read many times (not for frequent small updates).
- High throughput over low latency:

- Optimized for large, sequential batch reads rather than fast random access.
- Rack awareness:
- HDFS is aware of the physical rack location of DataNodes and places replicas across different racks to survive even a rack-level failure.

12.6 Working Process — Reading and Writing a File in HDFS

1. Write: The client contacts the NameNode, which identifies which DataNodes should store the file's blocks.
2. The client writes each block directly to the assigned DataNode, which then replicates the block to additional DataNodes (a 'pipeline' write) to reach the replication factor.
3. The NameNode records the block-to-DataNode mapping in its metadata.
4. Read: The client contacts the NameNode to get the block locations for the requested file.
5. The client reads each block directly from the nearest/most appropriate DataNode (parallel reads across blocks/nodes for speed).

12.7 Fault Tolerance in Action

If a DataNode holding a block fails, the NameNode detects the missing heartbeat, notices the block's replication factor has dropped below the target, and instructs another DataNode to create a new replica of that block from one of the surviving copies — automatically restoring full redundancy without data loss.

12.8 Advantages of HDFS

- Highly fault-tolerant through automatic block replication.
- Scales to store petabytes of data across thousands of inexpensive commodity machines.
- Optimized for high-throughput batch processing of very large files.

12.9 Disadvantages / Limitations

- Poor performance for small files — many small files waste NameNode metadata memory and reduce efficiency ('small file problem').
- Not designed for low-latency random access or frequent updates to existing files.
- The NameNode is historically a single point of failure in older Hadoop versions (mitigated in modern Hadoop via High Availability NameNode configurations).

12.10 Applications

- Storing massive log files and clickstream data for batch analytics.
- Serving as the underlying storage layer for Hive and Spark workloads.
- Long-term, cost-effective archival of large historical datasets.

MEMORY TIP

HDFS memory tip: 'NameNode knows WHERE, DataNode stores WHAT' — NameNode holds metadata/location info; DataNodes hold the actual data blocks.

EXAM TIP

For 'Explain in detail about HDFS' (asked May 2023), always include: (1) the NameNode/DataNode master-slave diagram, (2) the block + replication concept, (3) the write and read process steps, (4) at least 2 advantages and 2 limitations. This exact structure matches the expected 7-mark answer depth.

13. Hive

13.1 Introduction

Writing raw MapReduce Java code to query Big Data is slow and requires strong programming skills — Apache Hive was created to let analysts familiar with SQL query massive datasets stored in HDFS using a familiar, SQL-like language instead.

13.2 Definition

DEFINITION

Apache Hive: A data warehousing and SQL-like query infrastructure built on top of Hadoop, which allows users to query, summarize, and analyze large datasets stored in HDFS (or compatible storage) using a SQL-like language called HiveQL (HQL), which is automatically translated into underlying MapReduce, Tez, or Spark jobs.

13.3 Hive Architecture

Component	Role
HiveQL (HQL)	SQL-like query language used to write queries against Hive tables
Driver	Manages the lifecycle of a HiveQL query — compilation, optimization, and execution
Metastore	Stores metadata about Hive tables (schema, location, partitioning) — critically, NOT the actual data itself
Execution Engine	Translates the query plan into underlying jobs (MapReduce, Tez, or Spark) that run on the Hadoop cluster
HDFS	The underlying storage layer where the actual table data resides

User writes HiveQL query

```

graph TD
    User[User writes HiveQL query] --> Driver[Driver]
    Driver <-->|consults| Metastore[Metastore]
    Driver --> Engine[Execution Engine]
    Engine -->|translates to| Jobs[MapReduce / Tez / Spark job]
    Jobs --> HDFS[HDFS]
    HDFS -->|actual data blocks read/processed|
  
```

13.4 Key Concepts

- Tables:
- Hive organizes data into tables similar to a relational database, but the data itself physically resides as files in HDFS.
- Partitioning:
- Dividing a table into parts based on the value of a column (e.g., partitioning sales data by 'year' and 'month') so queries can skip irrelevant partitions, greatly speeding up performance.
- Bucketing:

- Further dividing data within a partition into a fixed number of files ('buckets') based on a hash of a column, improving join and sampling performance.
- Schema-on-read:
- Like Hadoop generally, Hive applies the table schema only at query time, not at load time.

13.5 Working Process — Running a Query in Hive

1. The user writes a HiveQL query (very similar to standard SQL: SELECT, WHERE, GROUP BY, JOIN, etc.).
2. The Hive Driver parses and compiles the query, consulting the Metastore to resolve table schema and location details.
3. The query optimizer creates an execution plan (leveraging partition pruning where applicable).
4. The Execution Engine translates this plan into one or more underlying MapReduce/Tez/Spark jobs.
5. These jobs run in a distributed fashion on the Hadoop cluster, reading the relevant data blocks directly from HDFS.
6. Results are returned to the user, often via the Hive command line, a JDBC/ODBC connection, or a BI tool.

13.6 Hive vs Traditional RDBMS (Common Comparison)

Aspect	Hive	Traditional RDBMS
Data volume	Petabyte scale	Typically gigabyte to low terabyte scale
Latency	High latency (batch-oriented, seconds to minutes)	Low latency (fast, near-instant for indexed queries)
Schema enforcement	Schema-on-read	Schema-on-write
Update support	Traditionally limited update/delete support (improving in modern Hive with ACID tables)	Full support for updates, deletes, transactions
Best for	Large-scale batch analytics	Transactional (OLTP) applications

13.7 Advantages of Hive

- Makes Big Data on Hadoop accessible to analysts already skilled in SQL, without needing to write MapReduce code.
- Scales to handle massive datasets that a traditional RDBMS cannot.
- Supports familiar concepts (tables, partitions, joins) that ease the transition from relational databases.

13.8 Disadvantages / Limitations

- High query latency compared to a traditional database — unsuitable for real-time/interactive lookups.
- Historically limited support for row-level updates and deletes (improving with modern Hive ACID features, but still not as robust as an RDBMS).
- Requires careful partitioning/bucketing design to achieve good performance — poor design leads to full-table scans.

13.9 Applications

- Ad-hoc, SQL-style batch analytics over massive log or clickstream datasets stored in HDFS.
- Data warehousing on top of a Hadoop-based data lake.
- ETL-style transformations expressed as HiveQL queries across huge datasets.

MEMORY TIP

Hive memory tip: 'Hive gives SQL a Hadoop Hive-mind' — it lets many distributed workers (like bees) jointly answer one SQL-style query over massive HDFS-stored data.

14. Apache Spark

14.1 Introduction

Apache Spark was developed to address MapReduce's biggest weakness — its slow, disk-based processing model — by performing computations in-memory wherever possible, making it dramatically faster for iterative algorithms (like machine learning) and interactive data analysis.

14.2 Definition

DEFINITION

Apache Spark: An open-source, distributed data processing engine designed for large-scale data analytics, offering fast, in-memory computation, along with unified support for batch processing, real-time streaming, SQL queries, machine learning, and graph processing, all within a single framework.

14.3 Why Spark Is Faster Than MapReduce

MapReduce: Read from disk -> Map -> WRITE to disk -> Read from disk -> Reduce -> Write to disk
(Multiple slow disk read/write cycles between stages)

Spark: Read from disk -> Map (in-memory) -> Reduce (in-memory) -> Write final result to disk
(Intermediate results stay IN MEMORY across stages -> much faster, especially for iterative workloads like machine learning that reuse the same data many times)

14.4 Core Concept — RDD (Resilient Distributed Dataset)

DEFINITION

RDD (Resilient Distributed Dataset): Spark's fundamental data structure — an immutable, fault-tolerant collection of elements that can be processed in parallel across a cluster, and can automatically recover from node failures by recomputing lost partitions using their recorded lineage (the sequence of transformations used to build them).

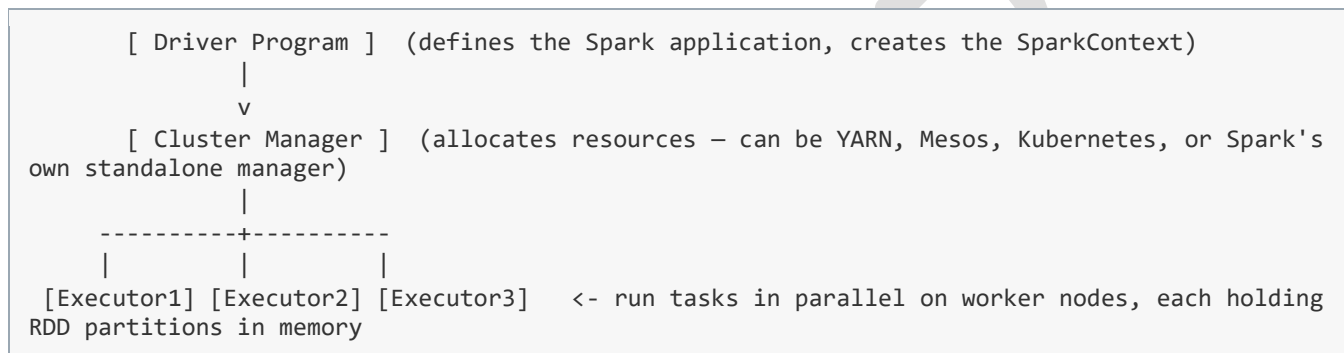
- Immutable:
- Once created, an RDD cannot be changed — transformations create new RDDs.
- Lazy Evaluation:
- Transformations (map, filter) are not executed immediately; they are recorded and only actually computed when an 'action' (like count() or collect()) requires a result — allowing Spark to optimize the overall execution plan.
- Fault Tolerant via Lineage:
- If a partition is lost, Spark can recompute just that partition using its recorded lineage graph, rather than needing physical replication like HDFS.

14.5 Spark Ecosystem / Components

Component	Purpose
Spark Core	The base engine — task scheduling, memory management, fault recovery, and RDD API

Component	Purpose
Spark SQL	Enables SQL-style querying over structured data, including a higher-level DataFrame API
Spark Streaming / Structured Streaming	Processes real-time, continuous data streams using the same engine as batch processing
MLlib	Spark's built-in scalable machine learning library (classification, regression, clustering, etc.)
GraphX	Library for graph-parallel computation (analyzing networks/relationships)

14.6 Spark Architecture



14.7 Working Process — Running a Spark Job

1. The Driver program defines the sequence of transformations (e.g., load data, filter, map, aggregate) on an RDD or DataFrame.
2. Because of lazy evaluation, nothing actually runs yet — Spark builds a logical execution plan (a Directed Acyclic Graph, DAG, of operations).
3. When an action (e.g., count(), collect(), save()) is called, Spark's scheduler converts the DAG into a set of stages and tasks.
4. The Cluster Manager allocates Executors (worker processes) across the cluster.
5. Tasks run in parallel on the Executors, processing their assigned data partitions primarily in memory.
6. Results are returned to the Driver or written to storage (e.g., back to HDFS or a data lake).

14.8 Spark vs Hadoop MapReduce (Very Common Comparison)

Aspect	Hadoop MapReduce	Apache Spark
Processing model	Disk-based, two-stage (Map, Reduce)	In-memory, multi-stage DAG-based
Speed	Slower, especially for iterative jobs	Typically 10-100x faster for iterative/in-memory workloads
Ease of use	Requires verbose Java MapReduce code	Rich, concise APIs in Python, Scala, Java, R; supports SQL
Real-time processing	Not natively supported (batch only)	Supports streaming via Structured Streaming
Machine learning support	Requires separate libraries (e.g., Mahout)	Built-in MLlib library

14.9 Advantages of Spark

- Dramatically faster than MapReduce for iterative and interactive workloads due to in-memory computation.
- Unified engine — batch, streaming, SQL, machine learning, and graph processing all in one framework.
- Rich, developer-friendly APIs in multiple languages (Python/PySpark, Scala, Java, R), plus SQL support.
- Can run on top of Hadoop (using HDFS/YARN) or independently, offering deployment flexibility.

14.10 Disadvantages / Limitations

- Requires significantly more memory (RAM) than disk-based MapReduce, which can be costly at very large scale.
- More complex to tune for optimal performance (memory management, partitioning strategy).
- Can be less cost-efficient than MapReduce for simple, one-off batch jobs that don't benefit from in-memory reuse.

14.11 Applications

- Real-time fraud detection and recommendation engines (Structured Streaming).
- Large-scale machine learning model training (MLlib).
- Interactive, exploratory big-data analysis via Spark SQL / DataFrames.
- ETL processing as a faster alternative to traditional MapReduce-based Hive jobs.

MEMORY TIP

Spark memory tip: 'Spark keeps data IN MEMORY, MapReduce keeps writing to DISK' — memory vs disk is the single biggest differentiator to state first in any comparison answer.

EXAM TIP

When Big Data processing tools are asked together (as in May 2024 Q8b), always present Hadoop → HDFS → Hive → Spark as a connected story: Hadoop is the foundation (storage=HDFS + old processing=MapReduce), Hive adds SQL-style querying on top, and Spark is the faster, unified, in-memory alternative processing engine — this narrative structure earns strong marks for showing how the pieces fit together.

15. Exam Focus & Quick Revision

15.1 Most Important Topics (Ranked by PYQ Frequency + Syllabus Risk)

1. File Formats — asked in BOTH 2023 and 2024; highest-priority topic in this unit.
2. HDFS — asked directly in 2023; very likely to repeat given its central role in the syllabus.
3. Data Analyst Ecosystem — asked directly in 2023; likely repeat candidate.
4. Big Data Processing Tools (Hadoop/Hive/Spark together) — asked in 2024; expect variations like 'explain Spark' or 'explain Hive' individually too.
5. Hive — appeared as a short-note option in 2023; prepare to full 7-mark depth as insurance.
6. NoSQL, Data Lakes, Data Marts, ETL & Data Pipelines, Types of Data, Sources of Data — NOT yet directly asked in the two available papers, but are explicit named syllabus items; treat these as high-risk 'due to appear' topics.

15.2 Expected Long-Answer Questions (7 / 14 Marks)

- What do you understand by Data Analyst Ecosystem? Explain with a diagram.
- Describe the different types of file formats in detail (asked verbatim in both years).
- Explain HDFS architecture and its working in detail.
- What is Hive? Explain its architecture and how a HiveQL query is executed.
- What is Apache Spark? Explain its architecture and why it is faster than MapReduce.
- What is Big Data? Explain the 5 V's and various Big Data processing tools in detail.
- Explain NoSQL databases and their types with examples.
- Differentiate between Data Warehouse, Data Mart, and Data Lake.
- Explain ETL and the concept of Data Pipelines with a diagram.
- Explain the different types of data with examples (structured, semi-structured, unstructured).

15.3 Expected Short-Answer / Definition Questions

- Define Big Data and list its key characteristics.
- What is the difference between a Data Lake and a Data Warehouse?
- Name any two NoSQL database types with an example system for each.
- What is the role of the NameNode in HDFS?
- Differentiate ETL and ELT.
- What is an RDD in Spark?
- Name any three columnar Big Data file formats.
- Define a Data Mart.

15.4 Frequently Repeated Concepts to Master First

- File format tables (text-based vs columnar Big Data formats) + row vs columnar diagram.
- HDFS NameNode/DataNode architecture + block replication + read/write process.
- The 5 V's of Big Data.
- Hadoop's four core components (HDFS, MapReduce, YARN, Common).
- Hive architecture diagram (Driver, Metastore, Execution Engine, HDFS).
- Spark's RDD concept + in-memory vs disk-based comparison with MapReduce.

- Data Warehouse vs Data Mart vs Data Lake distinctions.

15.5 Quick Revision Sheet

Concept	One-Line Definition / Key Point
Data Analyst Ecosystem	The full set of roles, tools, and processes from data source to business decision
Structured / Semi-structured / Unstructured	Rigid schema / tagged-but-flexible / no schema at all
Parquet / ORC	Columnar Big Data file formats optimized for analytical queries
Avro	Row-based binary format with embedded schema, ideal for streaming
Data Warehouse	Structured, schema-on-write repository for historical business reporting
Data Mart	Smaller, department-focused subset of a data warehouse
Data Lake	Raw, schema-on-read repository storing all data types at scale
NoSQL	Non-relational databases: document, key-value, column-family, graph
ETL	Extract, Transform, Load — moves and prepares data for a target repository
Data Pipeline	Automated, broader data-movement workflow (batch or streaming)
Big Data 5 V's	Volume, Velocity, Variety, Veracity, Value
Hadoop	Open-source framework for distributed storage (HDFS) + processing (MapReduce) on commodity hardware
HDFS	Distributed file system: NameNode (metadata) + DataNodes (data blocks, replicated 3x)
Hive	SQL-like querying (HiveQL) over Hadoop-stored data, translated into MapReduce/Spark jobs
Spark	Fast, in-memory, unified engine for batch, streaming, SQL, ML, and graph processing
RDD	Spark's immutable, fault-tolerant, distributed data structure with lineage-based recovery

End of Unit IV Study Guide. Cross-check each topic above against the latest official RGPV syllabus copy before your exam, as syllabi are occasionally revised.